

7-4 · MinIO·Qdrant 연동 · 실습 1.5H

MinIO → 임베딩 → Qdrant 적재 파이프라인 실습

문서를 넣어 두면 알아서 검색 가능해지는 상태로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

① 버킷 순회 → 자르기 → 임베딩 → upsert 를 ingest() 한 함수로 잇는다

② 재실행해도 포인트 수가 늘지 않는 것을 assert 로 확인한다

③ 검색 결과의 source 로 MinIO 의 원본 전문을 다시 읽어 온다

④ 문서가 줄었을 때 옛 조각이 남는 구멍을 재현하고 재적재로 막는다

emb 는 7-3 과 같은 임베딩 모델 객체를 그대로 — 차원이 달라지면 컬렉션을 새로 만들어야 합니다

일곱 단계 — ingest() 하나를 만들고, 깨뜨리고, 고친다

7-3 의 Qdrant(6333)·방금 만든 raw-docs 버킷·같은 emb 객체로 시작합니다

1. ingest()	2. 재실행	3. 일부러 실패	4. 역참조	5~7. 구멍·재적재
순회 → 자르기 → 임베딩 → upsert	before == after assert	uuid4 → 두 배	source 로 원본 읽기	줄어든 문서 전체 재색인

1단계

ingest() — 버킷의 .txt 를 훑어 lecture_kb 에 넣는다



`payload = {text, source: minio://raw-docs/..., chunk_index} · 컬렉션 lecture_kb · COSINE · DIM = len(emb.embed_query(...))`

확인 포인트: 문서별 「source → N chunks」가 찍히고 client.count 가 조각 총수와 같아야 한다

멱등 확인, 그리고 uuid4 로 바꿔 깨뜨려 본다

uuid5 — 재실행해도 그대로

before == after 를 assert 로 쟁다

- before = client.count(COLLECTION).count
- ingest() 한 번 더 → after
- assert before == after, "멱등하지 않습니다"

VS

uuid4 — 두 번째에 두 배

실습용 별도 컬렉션 idempotency_test 에서

- PointStruct(id=str(uuid.uuid4()), ...) — 매번 새 id
- ingest_bad() 두 번 → count 가 두 배
- 검색하면 같은 내용이 중복 · delete_collection(TMP)

검색 결과에서 원본을 역참조한다 — 카드함에서 창고로

1 `query_points(..., query=emb.embed_query(질문), limit=1)`
top.payload["text"][:60] 과 top.payload["source"] 를 출력

2 `source` 를 `minio:// · 버킷 · 키`로 쪼갬다
partition("minio://") → rest.partition("/") 로 bucket, key

3 `mc.get_object(bucket, key)` 로 원본 전체
출처가 `minio://raw-docs/notes/notes.txt` 로 찍히고 전문이 다시 읽힌다

카드 → 창고

source 로
원본 전문

카드함에서 찾아 창고의 원본을 꺼내는 흐름 완성 — 이론에서 그린 그림이 코드로

문서가 줄어들 때의 구멍을 재현한다

1 **count_of(SRC) — source 필터로 조각 수 세기**
`Filter(must=[FieldCondition(key="source", match=MatchValue(...))])`

2 **long_notes.txt 를 한 줄짜리 short.txt 로 덮어 올린다**
`fput_object(BUCKET, "notes/long_notes.txt", "short.txt")`

3 **ingest(prefix="notes/long_notes.txt") 다시**
변경 후 count 가 줄어들지 않는다 — 앞부분만 덮어쓰고 뒷조각이 남았다

증상

지워진 내용이
검색에 나온다

실무에서 가장 골치 아픈 종류의 버그 — 이론 시간의 「먹등이 완벽하지는 않다」가 이것

재적재로 고치고, 컬렉션째 다시 색인한다

reingest() — 지우고 다시 넣기

그 문서의 옛 조각을 모두 지우고 ingest(prefix=...)

- `client.delete(..., points_selector=FilterSelector(filter=...))`
- `return ingest(prefix=object_name)`
- 최종 `count_of(SRC)` 가 줄어든다 — 가장 단순하고 확실한 전략

VS

recreate_collection — 전체 재색인

`chunk_size 500 → 300` 으로 바꾸고 컬렉션째 새로

- `RecursiveCharacterTextSplitter(300, chunk_overlap=30)`
- `recreate_collection(COLLECTION, VectorParams(DIM, COSINE))`
- `ingest()` → 조각 수가 늘어난다 — 원본이 MinIO 에 있으니 자유롭다

이 1.5시간이 끝날 때 확인할 것

- 1 문서별 「source → N chunks」와 client.count 가 조각 총수와 같다
- 2 ingest() 를 다시 돌려도 before == after — assert 통과
- 3 uuid4 버전은 두 번째 실행에서 count 가 두 배가 된다
- 4 출처가 minio://raw-docs/notes/notes.txt 로 찍히고 원본 전문이 읽힌다
- 5 줄인 문서는 count 가 그대로 → reingest() 뒤에 줄어든다
- 6 chunk_size 300 으로 재색인하면 조각 수가 늘어난다

7-4 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

MinIO 문법 여덟 줄을 한 표로 정리하고, 7-5 Hybrid Search 를 위한 준비를 확인합니다

✓ MinIO·Qdrant 코드 요약표

✓ docker ps 로 qdrant·minio 둘 다 Up

✓ 체크리스트와 자주 겪는 오류 네 가지

✓ 7-5 준비: langchain · streamlit 설치

가져갈 것

lecture_kb 컬렉션에 포인트가 든 상태